Universidad de San Carlos de Guatemala — Centro Universitario de Occidente

Manual Técnico de REDXela

Arquitectura, concurrencia y operación de la simulación

Dany Noé de León Santizo — 201830006

Octubre de 2026

Índice

1. Introducción y alcance **1**
1.1 Propósito . . . 1
1.2 Problema y metas . . . 1
1.3 Actores y flujos . . . 1
1.4 Límites de la simulación . . . 2
1.5 Convenciones de lectura . . . 2
1.6 Correspondencia con el encargo . . . 2

2. Tecnologías y dependencias **2**
2.1 Plataforma de aplicación . . . 2
2.2 Bibliotecas Maven . . . 2
2.3 Concurrencia del JDK . . . 3
2.4 Herramientas para el manual . . . 3
2.5 Requisitos de entorno . . . 4
2.6 Separación de responsabilidades . . . 4

3. Requisitos de software e instalación **4**
3.1 Requisitos previos . . . 4
3.2 Obtener el código y ubicar el proyecto . . . 4
3.3 Compilación de la aplicación . . . 5
3.4 Herramientas del manual técnico . . . 5
3.5 Errores frecuentes de instalación . . . 5
3.6 Alcance de esta instalación . . . 6

4. Configuración y ejecución **6**
4.1 Configuración presente . . . 6
4.2 Inicio del sistema . . . 6
4.3 Uso inicial para observar el flujo . . . 7
4.4 Formularios y acciones . . . 7
4.5 Actualización del tablero . . . 7
4.6 Cierre de la aplicación . . . 7
4.7 Parámetros temporales . . . 8

5. Arquitectura y paquetes Java **8**
5.1 Organización general . . . 8
5.2 Frontera web . . . 9
5.3 Servicios . . . 9
5.4 Modelo y configuración . . . 10

5.5 Estado en memoria y consistencia . . . 10
5.6 Límites arquitectónicos . . . 10

6. Modelo de dominio y recursos **10**
6.1 Cuentas, catálogo y mercancía . . . 10
6.2 Orden y líneas . . . 11
6.3 Niveles de servicio . . . 11
6.4 Capacidades de operación . . . 11
6.5 Almacén, ubicaciones y lotes . . . 12
6.6 Resultados de recepción y reserva . . . 12

7. Recepción e inventario **12**
7.1 Separar entrada web de registro de existencias . . . 12
7.2 FIFO, capacidad y respuesta a presión . . . 12
7.3 Asignación física . . . 13
7.4 Reserva y consumo . . . 13
7.5 Reconsideración de órdenes . . . 14
7.6 Cierre y resultados pendientes . . . 14

8. Gestión de pedidos y políticas de planificación **14**
8.1 Ingreso manual y cuentas . . . 14
8.2 Generador automático . . . 15
8.3 Cola elegible . . . 15
8.4 Políticas disponibles . . . 15
8.5 Estados y recuperación . . . 17
8.6 Métricas de espera . . . 17
8.7 Volumen de trabajo . . . 17

9. Concurrencia, recursos compartidos y prevención de deadlock **17**
9.1 Procesamiento en paralelo . . . 17
9.2 Semáforos de capacidad . . . 18
9.3 Orden total de adquisición . . . 18
9.4 Recursos que compiten en la operación . . . 18
9.5 Monitores para estado compuesto . . . 18
9.6 Límites de las garantías . . . 20

10. Demostración de conflicto y manejo de errores **20**
10.1 Objetivo didáctico del conflicto . . . 20
10.2 Aislamiento y expiración . . . 21
10.3 Validación de formularios . . . 21

10.4 Errores de procesamiento 21
10.5 Consideraciones de seguridad y operación 22

11. Dashboard, métricas y comparación de políticas 22
11.1 Vista principal 22
11.2 Visibilidad disponible 22
11.3 Datos y cálculo de las estadísticas 23
11.4 Comparación controlada 23
11.5 Lectura de resultados 23
11.6 Interpretación operacional 24

12. Ciclo de vida, límites y estado de verificación 24
12.1 Inicio de hilos y ejecutores 24
12.2 Cierre del generador 24
12.3 Cierre del consumidor de recepción 24
12.4 Cierre de despacho y trabajadores 25
12.5 Persistencia y límites de operación 25
12.6 Estado de verificación 25
12.7 Limitaciones que deben exponerse 26

1. Introducción y alcance

1.1 Propósito

REDXela es una simulación web de un centro de distribución de comercio electrónico. Su propósito académico es hacer visibles problemas que en una operación logística aparecen cuando varias órdenes, recepciones y áreas compiten por capacidad limitada. El proyecto aplica hilos, colas, exclusión mutua, semáforos, planificación e inventario, y ofrece una pantalla para observar el estado resultante.

El sistema no es un gestor comercial ni un sistema de almacén persistente. Los clientes, órdenes, lotes, ubicaciones, errores y resultados de comparación viven en memoria durante la ejecución. El alcance es el de una simulación de una sola instancia y un solo proceso Java. No se documentan integraciones externas ni persistencia porque no forman parte de la configuración actual.

1.2 Problema y metas

El planteamiento de la asignación describe llegadas variables de pedidos, recursos operativos compartidos y almacenamiento finito. La aplicación representa las partes principales de ese problema:

- registrar pedidos manuales y generados automáticamente;
- priorizar pedidos con tres políticas seleccionables;
- reservar existencias antes del trabajo y registrar consumo al terminar;
- recibir mercadería por una cola FIFO separada de la operación web;
- limitar los recursos con semáforos y proteger el inventario compartido;
- estimar el trabajo usando el número de unidades;
- mostrar colas, pedidos activos, existencias, recursos y resultados en una interfaz que se actualiza periódicamente;
- presentar un escenario controlado de conflicto y permitir una resolución explícita.

Estas metas se corresponden con el código de `api/src/main/java/com/sopes/api`. La interfaz está en `api/src/main/resources/templates/dashboard.html`. La descripción de una función en este manual significa que existe en el código inspeccionado; no significa que haya sido validada mediante pruebas manuales o una ejecución de extremo a extremo.

1.3 Actores y flujos

El operador de la simulación usa un navegador para consultar el tablero, crear órdenes, ingresar recepciones, iniciar o detener el generador, cambiar la política de cola y resolver la demostración. Los servicios Spring coordinan cada acción en el proceso Java. Los hilos de fondo realizan el consumo de recibos, despacho de órdenes y trabajo simulado.

Una orden puede quedar en espera de inventario, en cola elegible, en proceso, bloqueada o completada. Una recepción enviada correctamente pasa a la cola de recepción, después actualiza inventario y puede volver elegibles órdenes que esperaban existencias. La demostración de conflicto tiene estado propio; no representa una orden real ni toma los permisos productivos.

1.4 Límites de la simulación

La duración del trabajo se aproxima a 0.5 segundos por unidad; no modela tiempos distintos por tipo de producto, tiempos de transporte reales, turnos ni costos. El conjunto de mercancías y clientes está precargado en servicios Java. La capacidad del almacén está fijada en 1,000 unidades, dividida en cuatro ubicaciones de 250 unidades.

La comparación de políticas usa una fotografía de la cola elegible, cuatro trabajadores virtuales y el mismo costo fijo por unidad para las tres políticas. No incluye las restricciones de recursos compartidos ni la interferencia de nuevos pedidos durante la proyección. Por ello ayuda a comparar el efecto del ordenamiento sobre una carga capturada, pero no predice el rendimiento real.

La aplicación no incluye autenticación, roles, base de datos, configuración distribuida, monitoreo externo ni recuperación después de reiniciar. El manual técnico explica el diseño existente y sus límites, no propone que esas capacidades estén implementadas.

1.5 Convenciones de lectura

Los nombres de clases y rutas se escriben como aparecen en el proyecto. Las rutas relativas de comandos se ejecutan desde la raíz del repositorio salvo que se indique otra cosa. Los diagramas describen relaciones de alto nivel; cuando el estado es explícitamente simulado, el diagrama lo indica. La sección final separa verificación de código de comportamiento efectivamente probado.

1.6 Correspondencia con el encargo

El encargo pide una interfaz para observar recursos, órdenes, progreso, lista de espera, estadísticas, entradas manuales y conflicto. También solicita un manual técnico que explique infraestructura y decisiones de diseño y un manual de usuario. Este documento cubre únicamente el manual técnico. La interfaz y el procesamiento se describen con las diferencias entre lógica productiva y demostración simulada; no se atribuyen capacidades que no aparezcan en las clases revisadas.

2. Tecnologías y dependencias

2.1 Plataforma de aplicación

El backend está escrito en Java y usa Spring Boot. La versión del lenguaje se fija en `25` mediante la propiedad `java.version` de `api/pom.xml`. El POM declara Spring Boot `4.1.2` como parent. Spring MVC procesa solicitudes web y Thymeleaf renderiza la vista de servidor. La plantilla utiliza Bootstrap 5.3.8 desde un CDN para distribución visual; por tanto, la carga de esos estilos en el navegador depende de que el CDN sea accesible.

La aplicación no declara una base de datos ni un ORM. Tampoco incorpora mensajería externa: la comunicación de recepción utiliza `ArrayBlockingQueue` dentro del mismo proceso. Los servicios son beans Spring (`@Service`) y la configuración de recursos se construye en `ResourceConfiguration`.

2.2 Bibliotecas Maven

Las dependencias de producción explícitas son:

Dependencia	Uso en REDXela
`spring-boot-starter-webmvc`	Controlador MVC, solicitudes HTTP y respuesta JSON del estado vivo.
`spring-boot-starter-thymeleaf`	Renderizado de `dashboard.html` desde el modelo de Spring.
Starter de pruebas MVC	Está declarado con alcance `test`; no participa al ejecutar la aplicación.

El proyecto usa las dependencias administradas por el parent de Spring Boot. No se deben inferir versiones independientes adicionales a las del POM y su gestión de dependencias.

2.3 Concurrencia del JDK

La sincronización principal proviene de `java.util.concurrent` y de monitores de Java:

Mecanismo	Ubicación	Función documentada
`Thread`	Generador, despachador y consumidor	Ejecutar tareas periódicas o de fondo.
`ThreadPoolExecutor`	`OrderProcessingService`	Ejecutar hasta cuatro órdenes al mismo tiempo, con cola de tareas acotada.
`ArrayBlockingQueue` / `BlockingQueue`	`ReceivingService` y trabajadores	Mantener límites explícitos a las tareas en espera.
`Semaphore`	`SharedResourceService`	Limitar unidades disponibles de cada recurso compartido.
`synchronized`	Inventario, cola de órdenes, registro de clientes y conflicto	Serializar transiciones del estado interno.
`AtomicBoolean`, `AtomicInteger`	Recepción	Coordinar estado de ejecución y conteo de recibos en curso.
`CopyOnWriteArrayList`	Errores y resultados recientes	Exponer snapshots sencillos de colecciones pequeñas.

No se incorpora un framework adicional de concurrencia. El uso de semáforos, monitores, hilos y una cola de bloqueo es parte visible de la solución y no se simula con temporizadores de interfaz.

2.4 Herramientas para el manual

El generador de `docs/tecnico/generar-pdf.sh` combina capítulos Markdown mediante Pandoc y XeLaTeX. Python 3 procesa los bloques Mermaid y Mermaid CLI (`mmdc`) genera los diagramas como PDF vectorial. Esta cadena documental es independiente del build Maven. La generación requiere que esos comandos estén disponibles en `PATH`, además de una distribución TeX con los paquetes usados en `plantilla.tex`.

2.5 Requisitos de entorno

Para desarrollar o ejecutar la aplicación se requiere un JDK compatible con Java 25 y Maven. El wrapper Maven se encuentra en `api/mvnw`; se puede usar el Maven instalado en la máquina. Para compilar el manual se necesitan las herramientas del apartado 2.4. El sistema no requiere Node para iniciar REDXela; Node/Chromium son requisitos indirectos de Mermaid CLI al renderizar diagramas.

Las versiones instaladas deben comprobarse en la máquina de entrega. Este manual registra las versiones declaradas por el proyecto, no afirma que cualquier instalación local haya compilado o generado el PDF. La disponibilidad efectiva de Bootstrap también depende de la conexión del navegador al CDN indicado por la plantilla HTML.

2.6 Separación de responsabilidades

Spring se ocupa del ciclo de vida de los servicios y del enrutamiento HTTP; Java aporta mecanismos de sincronización y ejecución. Thymeleaf genera el documento inicial y JavaScript consulta el endpoint de estado para refrescar secciones. Pandoc, XeLaTeX y Mermaid CLI son herramientas para documentar, no dependencias de la aplicación. Esta separación permite ejecutar el simulador sin la cadena editorial y elaborar el manual sin arrancar los servicios de negocio.

3. Requisitos de software e instalación

3.1 Requisitos previos

El repositorio incluye el proyecto Spring Boot en `api/`. Para compilarlo y ejecutarlo, instala un JDK 25 y Maven, o utiliza el wrapper incluido si su distribución puede descargar el Maven configurado. Comprueba las versiones antes de empezar:

```
java -version
mvn -version
```

En Windows el wrapper se llama `api\mvnw.cmd`; en sistemas tipo Unix, `api/mvnw`. Si se usa `mvn`, la versión instalada debe aceptar el POM del proyecto. El proyecto define Java 25 y no fija un perfil de ejecución adicional.

La interfaz depende del navegador moderno y de acceso al CDN de Bootstrap 5.3.8. Sin el CDN, el HTML y las acciones de Spring siguen siendo la base de la interfaz, pero el estilo Bootstrap puede no cargarse. La aplicación no requiere instalación local de Node, Chromium, Pandoc o TeX para ejecutarse.

3.2 Obtener el código y ubicar el proyecto

Clona o extrae el repositorio con la estructura íntegra. Desde su raíz, confirma que existen el POM y la clase de entrada:

```
test -f api/pom.xml
test -f api/src/main/java/com/sopes/api/ApiApplication.java
```

No ejecutes los comandos Maven desde `docs/tecnico`; el proyecto Maven está en `api/`. El `application.properties` actual establece únicamente el nombre de aplicación, por lo que el puerto y demás opciones conservan los valores predeterminados de Spring Boot mientras no se proporcionen parámetros externos.

3.3 Compilación de la aplicación

Con Maven instalado, compila desde `api/`:

```
cd api
mvn compile
```

El equivalente con el wrapper en Linux es `./mvnw compile`; en Windows puede usarse `mvnw.cmd compile`. La primera ejecución puede necesitar conexión a repositorios Maven para descargar dependencias. El resultado de compilación queda en `api/target/`, que es un directorio generado y no forma parte del contenido del manual.

Una compilación exitosa confirma que Maven resolvió y compiló las fuentes bajo la configuración activa. No demuestra por sí sola que las rutas de la interfaz, los ciclos concurrentes o los cierres hayan sido probados en ejecución.

3.4 Herramientas del manual técnico

La generación del PDF requiere:

Herramienta	Papel
Pandoc	Convierte y ensambla Markdown con la plantilla LaTeX.
XeLaTeX	Compone el PDF con tipografía Unicode y español.
Python 3	Extrae bloques Mermaid y prepara archivos temporales.
Mermaid CLI (`mmdc`)	Renderiza diagramas a PDF vectorial.
Node.js y Chromium	Dependencias operativas usuales de Mermaid CLI.

El README especifica paquetes TeX Live usados por la plantilla: XeTeX, paquetes LaTeX adicionales y fuentes recomendadas. La configuración local debe incluir `fvextra`, `titlesec`, `enumitem`, `microtype`, `fancyhdr`, `booktabs` y los demás paquetes estándar referenciados en `plantilla.tex`; normalmente se obtienen mediante los paquetes TeX Live mencionados.

No se ha generado este PDF en el estado descrito aquí porque el comando `mmdc` no está disponible en el entorno de revisión. Instalar o configurar esa herramienta es un paso pendiente, no una función de REDXela.

3.5 Errores frecuentes de instalación

Si Maven informa una versión de Java incompatible, revisa el `JAVA_HOME` de la terminal y `java -version`; el POM selecciona Java 25. Si no se descargan dependencias, comprueba conectividad y configuración del repositorio Maven. Si Spring no puede iniciar por puerto ocupado, configura `server.port` temporalmente como argumento, por ejemplo `--server.port=8081`.

Si el manual informa que falta una herramienta, valida su presencia con `command -v pandoc`, `command -v xelatex`, `command -v python3` y `command -v mmdc`. Cuando Mermaid CLI exista pero no arranque el navegador, revisa la disponibilidad de Chromium y las dependencias del sistema que requiere esa instalación. El generador reporta la herramienta faltante; no la instala.

3.6 Alcance de esta instalación

La aplicación es autocontenida en una JVM y no requiere servicios de base de datos ni broker. La información se reinicia al terminar el proceso, y cada instancia tiene su propio inventario, cola y catálogo en memoria. Para entregar el proyecto, debe conservarse el código fuente, este manual técnico y el manual de usuario separado que exige la asignación; el manual de usuario no se incluye en este documento.

4. Configuración y ejecución

4.1 Configuración presente

`api/src/main/resources/application.properties` contiene `spring.application.name=api`. No hay configuración de base de datos, perfiles Spring, puerto personalizado, archivos de secretos ni valores externos para capacidades. Las capacidades logísticas y ubicaciones se establecen en código por medio de `ResourceConfiguration` y `WarehouseConfiguration`.

El servidor usa el puerto predeterminado de Spring Boot, 8080, mientras no se cambie mediante argumentos o configuración del entorno. Para una sesión de laboratorio que necesite otro puerto, puede pasarse como parámetro de Spring Boot; no es un cambio persistente del proyecto:

```
cd api
mvn spring-boot:run
```

También puede ejecutarse el artefacto empaquetado si se generó previamente:

```
mvn package
java -jar target/api-0.0.1-SNAPSHOT.jar
```

La versión y nombre del artefacto pueden variar si se modifican las propiedades del POM. En la configuración inspeccionada, `groupId` es `com.sopes`, `artifactId` es `api` y la versión es `0.0.1-SNAPSHOT`.

4.2 Inicio del sistema

Al arrancar la clase `ApiApplication`, Spring crea los servicios y publica el contexto web. `OrderProcessingService.start()` inicia un hilo despachador durante `@PostConstruct`; `ReceivingService.start()` inicia el consumidor de la cola de recepción. El generador automático permanece detenido hasta que el usuario lo inicia desde el tablero. El servicio de conflicto tiene un planificador daemon para expirar una demostración activa.

Abre `http://localhost:8080/` en un navegador. La ruta raíz muestra el tablero Thymeleaf y usa los catálogos y estado disponibles en memoria. No existe una pantalla de inicio de sesión. La aplicación no documenta controles de acceso.

4.3 Uso inicial para observar el flujo

El estado comienza con inventario vacío. Una manera directa de observar órdenes procesables consiste en enviar primero una recepción desde el formulario de la interfaz y esperar el resultado de inventario; luego crear una orden con un producto que tenga existencias. También puede iniciar el generador, pero al crear pedidos sin stock disponible estos quedan en `WAITING_FOR_STOCK`.

La recepción se acepta en una cola acotada y la respuesta web indica que quedó encolada. El tablero actualiza el recuento de pendientes y la lista de resultados. Al almacenar stock, el servicio vuelve a revisar órdenes que esperaban existencias. La asignación de ubicaciones puede dividir un lote entre varias ubicaciones disponibles.

Al registrar órdenes, se puede elegir una de las tres políticas de cola. La política afecta el orden en que el despachador reclama órdenes elegibles, pero no selecciona órdenes que continúan esperando inventario. Las cinco clases de servicio se guardan en cada orden y se usan para prioridad y evaluación del objetivo de espera.

4.4 Formularios y acciones

El formulario de pedidos permite seleccionar una cuenta predefinida o crear/buscar una cuenta por correo, servicio, productos de catálogo y cantidades. El formulario de recepción acepta producto del catálogo y unidades recibidas. El generador permite establecer un intervalo entre 1 y 3,600 segundos y detenerlo. El control de política cambia la cola; el botón de comparación captura los pedidos en espera y ejecuta una proyección común.

Las operaciones de conflicto están disponibles de forma separada: iniciar, elegir operación ganadora o cancelar. La demostración está etiquetada como simulada porque su estado usa semáforos aislados y no representa recursos u órdenes productivas.

4.5 Actualización del tablero

Después de cargar la vista, JavaScript consulta `/dashboard/state` cada 2.5 segundos y reemplaza el contenido de las zonas dinámicas. El refresco no recarga el documento completo, por lo que normalmente conserva el formulario que el usuario está editando. Si una consulta falla, el código deja el contenido actual y vuelve a intentar en el siguiente ciclo; no muestra un aviso específico de fallo de actualización.

La ruta `/dashboard/state` devuelve una estructura JSON con estadísticas, colas, órdenes activas y completadas, recursos, inventario, recepciones, conflicto, comparación y errores recientes. La ruta es una lectura de estado; las acciones se envían a las rutas POST definidas en `DashboardController`.

4.6 Cierre de la aplicación

Detén la aplicación con la señal normal de terminación del proceso o con `Ctrl+C` en la terminal. Spring invoca los métodos `@PreDestroy` de los servicios. La sección 12 detalla sus esperas y las limitaciones que conviene considerar. Como el estado es volátil, un cierre termina la sesión y no conserva órdenes ni stock para el siguiente inicio.

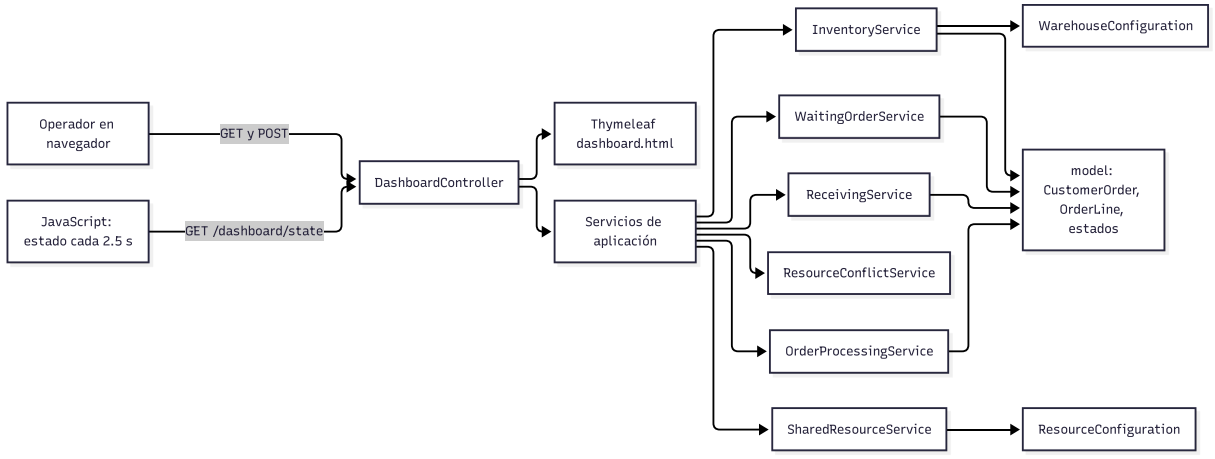


Figura 1: 5. Arquitectura y paquetes Java

4.7 Parámetros temporales

Las opciones suministradas por argumento no quedan guardadas en los archivos del proyecto. Por ejemplo, el puerto puede ser cambiado así:

```
cd api
mvn spring-boot:run -Dspring-boot.run.arguments=--server.port=8081
```

El intervalo del generador, en contraste, se configura desde el tablero al iniciarlo y pertenece a la instancia viva. No hay archivo de configuración para ajustar los límites de los semáforos o los cuatro espacios físicos; modificar esos valores requiere editar las clases de configuración.

5. Arquitectura y paquetes Java

5.1 Organización general

La aplicación sigue una organización por responsabilidades dentro de `com.sopes.api`. La clase `ApiApplication` habilita Spring Boot. El paquete `controller` contiene la frontera HTTP y prepara el modelo que Thymeleaf presenta. `service` agrupa el estado y las reglas operativas. `model` contiene registros, enumeraciones y objetos de dominio. `config` aporta capacidades compartidas a través de beans Spring.

La arquitectura es monolítica: los componentes comparten memoria y ciclo de vida dentro de una JVM. Las colas de Java coordinan subsistemas internos, pero no son un broker distribuido. No hay acceso remoto entre microservicios ni una API pública documentada para integraciones externas.

El diagrama muestra dependencias funcionales, no una secuencia temporal completa. `DashboardController` consulta varios servicios para construir la pantalla y también recibe las acciones de formularios. La ejecución de las tareas de fondo continúa sin una solicitud HTTP abierta.

5.2 Frontera web

`DashboardController` expone `GET /` para el documento inicial y `GET /dashboard/state` para el snapshot JSON. Los endpoints POST atienden órdenes, recepciones, políticas, generador y conflicto. Los formularios usan redirección al tablero y atributos flash para mostrar mensajes en la siguiente carga. Las rutas principales son específicas de la interfaz, no una API REST de recursos CRUD.

`dashboard.html` combina expresiones Thymeleaf para el render inicial con JavaScript para refresco parcial. El navegador no mantiene estado autoritativo: la fuente de verdad está en los servicios de la instancia. El refresco periódico lee el backend, mientras que los formularios transmiten comandos.

5.3 Servicios

Los servicios tienen responsabilidades diferenciadas:

Servicio	Responsabilidad principal
`OrderIntakeService`	Validar productos y cantidades recibidas del formulario y construir líneas.
`CustomerRegistry`	Registrar, listar y resolver cuentas por identificador o correo normalizado.
`ProductCatalog`, `MerchandiseCatalog`	Catálogos precargados de productos y requerimientos logísticos.
`WaitingOrderService`	Mantener órdenes, ordenar cola, reservar stock al reclamar y calcular estadísticas.
`OrderGeneratorService`	Generar llegadas automáticas en un hilo controlable.
`OrderProcessingService`	Despachar órdenes y ejecutar trabajo en un pool limitado.
`ReceivingService`	Aceptar solicitudes en FIFO, consumirlas y publicar resultados recientes.
`InventoryService`	Controlar lotes, cantidades disponibles, reservas y ubicación física.
`SharedResourceService`	Administrar semáforos y leases de recursos productivos.
`SchedulingComparisonService`	Proyectar políticas sobre la misma captura de órdenes.
`ResourceConflictService`	Modelar y resolver el escenario demostrativo aislado.

La tabla resume las clases principales, no todas las operaciones ni cada llamada entre ellas. Por ejemplo, el controlador no modifica las colecciones del inventario directamente; delega la recepción y consulta a los servicios.

5.4 Modelo y configuración

Los tipos de `model` incluyen `Customer`, `CustomerOrder`, `OrderLine`, `Product`, `MerchandiseType`, `StockLot`, `WarehouseLocation`, `SharedResource`, reservas, resultados y enumeraciones. Muchos son `record`, lo que hace explícitos los datos transportados entre servicios. Las transiciones de la orden generan nuevas instancias con los tiempos y estado actualizados.

`ResourceConfiguration` declara el catálogo de recursos y sus capacidades. `WarehouseConfiguration` declara capacidad de 1,000 unidades y cuatro ubicaciones. Los catálogos de mercancía y producto están en servicios porque son listas inmutables precargadas, no configuración externa.

5.5 Estado en memoria y consistencia

Cada bean singleton mantiene el estado de su área durante la vida del contexto Spring. `WaitingOrderService` mantiene el registro central de órdenes; `InventoryService` mantiene existencias y reservas; `ReceivingService` mantiene la cola y un historial acotado de resultados. No se replica deliberadamente una segunda fuente persistente. La interfaz solicita snapshots construidos desde esos servicios.

La sincronización no es uniforme en toda la aplicación porque protege datos diferentes: métodos `synchronized` cubren transiciones compuestas de pedidos e inventario, mientras semáforos coordinan cantidades finitas de recursos. `CopyOnWriteArrayList` se utiliza en listas pequeñas que se leen con frecuencia y se actualizan con menos frecuencia. El capítulo 9 explica las decisiones de concurrencia.

5.6 Límites arquitectónicos

La estructura facilita una explicación por capas, pero todos los servicios comparten proceso, memoria y recursos de CPU. Reiniciar la JVM pierde el estado. La disponibilidad de una única instancia no ofrece alta disponibilidad y el acceso al estado JSON no agrega autenticación. Estas propiedades son límites deliberados de una simulación universitaria pequeña, no garantías de un sistema productivo.

6. Modelo de dominio y recursos

6.1 Cuentas, catálogo y mercancía

`CustomerRegistry` inicia con tres cuentas de ejemplo: Cliente demostración, Comercio Central y Tienda Xelajú. Un usuario puede seleccionar una cuenta o enviar nombre y correo. El registro normaliza el correo quitando espacios externos y pasando a minúsculas, y reutiliza la cuenta que ya corresponda a ese correo. El identificador generado tiene prefijo `CUS-`.

`ProductCatalog` define seis SKU inmutables y asocia cada producto con un `MerchandiseType`. `MerchandiseCatalog` define seis categorías: Frágil, Pesada, Regulada, Electrónica, Voluminosa y Paletizada. El tipo determina qué semáforos debe adquirir el procesador; no cambia el nivel de servicio ni la cantidad de inventario requerida. La lista en código cumple el mínimo de cinco clases diferentes, pero no dispone de mantenimiento desde la interfaz.

Las combinaciones son distintas por tipo. Voluminosa y Paletizada comparten los dos recursos Montacargas y Encargados de bodega, lo que constituye una combinación que puede competir en procesamiento normal. Electrónica requiere el escáner compartido y empaque. La lista exacta se

puede consultar también en la tabla de tipos de mercancía del dashboard.

6.2 Orden y líneas

`CustomerOrder` es un registro con ID, cliente, servicio, líneas, estado, marcas de tiempo, progreso y métricas de espera. Cada `OrderLine` contiene código y nombre del producto, tipo de mercancía y cantidad positiva. `getTotalUnits()` suma las líneas; ese total sirve como base de trabajo y de estimación de duración.

Las marcas separan creación, entrada a cola, inicio y finalización. Una orden sin existencias entra a `WAITING_FOR_STOCK` y todavía no tiene marca de entrada en la cola procesable. Cuando el inventario permite reactivarla, el servicio asigna el momento de entrada a cola. La espera medida se calcula entre esa marca y el inicio registrado por el trabajador; el objetivo se compara con el objetivo del nivel contratado.

Los estados disponibles son `WAITING_FOR_STOCK`, `QUEUED`, `PROCESSING`, `BLOCKED`, `COMPLETED` y `CANCELLED`. En la implementación revisada no se encontró una operación de cancelación expuesta por el controlador; la enumeración no demuestra que la cancelación se encuentre implementada como flujo.

6.3 Niveles de servicio

Nivel	Nombre	Objetivo de espera configurado
1	Exprés	0 minutos
2	Prioritario	10 minutos
3	Estándar	30 minutos
4	Programado	60 minutos
5	Económico	120 minutos

En Exprés, cero minutos es una interpretación estricta del objetivo de “casi instantáneo” de la asignación. El sistema mide el resultado observado; no garantiza que se cumpla. La política de prioridad usa el número menor como mayor prioridad. Las demás políticas no eliminan la métrica objetivo: solo cambian el orden de reclamo.

6.4 Capacidades de operación

`ResourceConfiguration` define las unidades de semáforo. Cada orden reúne los códigos requeridos por sus líneas; duplicados se eliminan antes de adquirir. Cuando una orden tiene líneas de distintos tipos, necesita la unión de los recursos de esos tipos.

Recurso compartido	Capacidad
Estaciones de empaque (`PACKING`)	6
Áreas de carga (`LOADING`)	2
Encargados de bodega (`WAREHOUSE_WORKER`)	10
Montacargas (`FORKLIFT`)	3
Control de calidad (`QUALITY_CONTROL`)	4
Escáner de códigos (`BARCODE_SCANNER`)	1

Los valores expresan unidades simultáneas permitidas, no cantidades de personas u objetos modeladas individualmente. `SharedResourceService` crea un semáforo justo para cada capacidad y registra los identificadores que mantienen leases para reportar uso en el tablero.

6.5 Almacén, ubicaciones y lotes

`WarehouseConfiguration` fija 1,000 unidades repartidas entre A-01, A-02, B-01 y B-02, con 250 unidades por ubicación. Una recepción crea un lote por cantidad aceptada y ocupa ubicaciones en el orden declarado hasta agotar el lote o el espacio. Así, un solo lote puede aparecer dividido entre ubicaciones no contiguas.

El inventario distingue existencias físicas (`storedUnits`), existencias disponibles para nuevos pedidos y existencias reservadas por órdenes en proceso. Reservar no libera capacidad física; completar consume unidades y reduce ocupación; liberar una reserva por interrupción devuelve unidades a disponibilidad. La vista por producto presenta disponible, reservado, almacenado y asignación actual por ubicación.

6.6 Resultados de recepción y reserva

`ReceiptRequest` identifica producto, mercancía y unidades solicitadas. `ReceiptResult` registra identificador del recibo, solicitado, aceptado, estado, mensaje, fecha de procesamiento y reparto por ubicación. Los estados distinguen encolado, aceptado, parcialmente aceptado, rechazado y fallido. Una solicitud encolada no equivale aún a mercancía almacenada; el resultado posterior es la confirmación de inventario.

7. Recepción e inventario

7.1 Separar entrada web de registro de existencias

Una recepción no cambia directamente las colecciones internas del almacén desde el controlador. `DashboardController` valida los campos, busca el SKU en `ProductCatalog`, construye un `ReceiptRequest` y lo entrega a `ReceivingService`. El servicio de recepción genera un ID y trata de encolar la solicitud. El consumidor independiente adquiere el escáner y delega el registro atómico a `InventoryService`.

La separación evita que el tiempo de almacenamiento bloquee el hilo que responde al formulario. La cola `ArrayBlockingQueue` tiene capacidad 100. `submit` espera como máximo 250 milisegundos para ofrecer el recibo. Si no hay espacio disponible, lo rechaza explícitamente; una recepción aceptada queda en FIFO y no se descarta en silencio. El contador visible combina los elementos en cola y el recibo actualmente en procesamiento.

El diagrama omite los intervalos de espera y las excepciones. Si la adquisición del escáner se interrumpe, el recibo aceptado obtiene estado fallido con mensaje. Las excepciones de ejecución también producen un resultado fallido; al liberar el lease, el escáner vuelve a quedar disponible.

7.2 FIFO, capacidad y respuesta a presión

La estructura de cola conserva el orden de inserción. La capacidad finita impide acumular un número ilimitado de objetos en memoria. Cuando la cola está llena durante la ventana de oferta, el formulario recibe un mensaje de rechazo y debe reenviarse de forma consciente. Esto evita bloquear

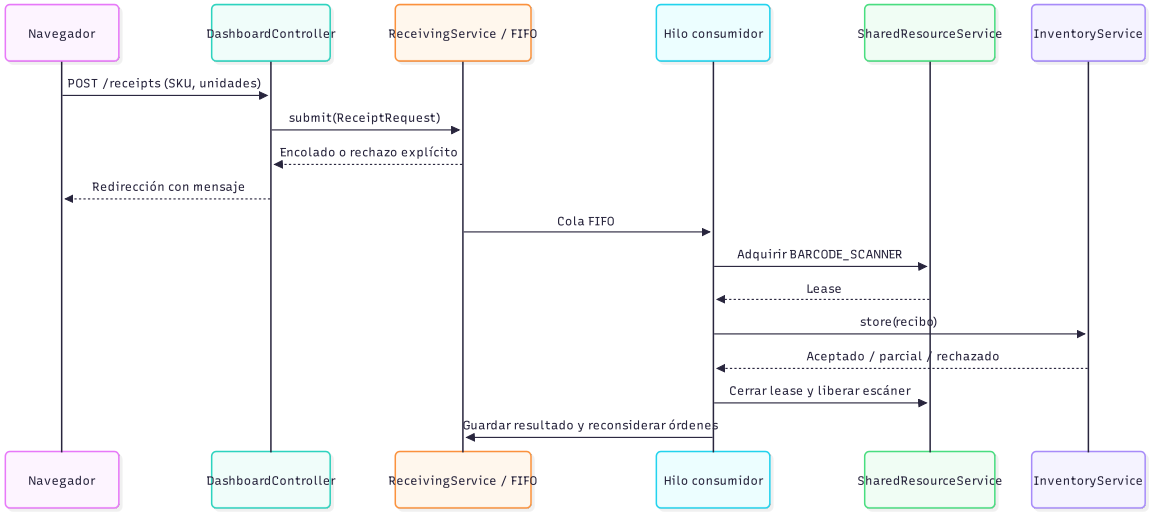


Figura 2: 7. Recepción e inventario

indefinidamente la solicitud HTTP y no clasifica como aceptado un elemento que no entró.

Es importante distinguir tres situaciones: aceptación para cola, procesamiento por consumidor y aceptación de unidades por almacén. Una solicitud puede haberse encolado correctamente y luego terminar parcialmente aceptada por falta de capacidad o fallida por una excepción/cierre. El tablero muestra resultados recientes, no un historial persistente; el historial conserva hasta 200 entradas en memoria.

7.3 Asignación física

El almacén admite 1,000 unidades: cuatro ubicaciones de 250. `InventoryService.store` es sincronizado y limita la cantidad aceptada según el espacio global restante. Recorre las ubicaciones configuradas y asigna cada porción en el espacio libre. Un recibo que no cabe completo puede almacenar una parte y reportar el número exacto y las ubicaciones de esa parte; si no cabe ninguna unidad, se rechaza.

Las estructuras de inventario registran lotes originales, asignación física por lote y ubicación, disponibilidad por lote y ubicación, reservas activas y ocupación total. Las consultas reconstruyen los lotes con cantidades físicas vigentes, de modo que el consumo no deja visible un lote agotado. No se reasigna stock entre ubicaciones: el consumo reduce las ubicaciones donde se reservó.

7.4 Reserva y consumo

Al reclamar una orden, `InventoryService.reserve` valida y descuenta la cantidad requerida bajo el mismo monitor. La reserva captura exactamente los lotes y ubicaciones comprometidos. Mientras la orden trabaja, dichas unidades siguen ocupando el almacén pero ya no están disponibles a otras órdenes. Al completar, `complete` retira la reserva y reduce unidades físicas y ocupación. Si se interrumpe el procesamiento o se necesita recuperar la orden, `release` reintegra la disponibilidad, sin repetir una reserva ya cerrada.

Este ciclo protege contra dos órdenes que intenten apropiarse simultáneamente de las mismas

unidades. `WaitingOrderService` y `InventoryService` coordinan la reclamación y el registro del estado. Los métodos del inventario son sincronizados para que las cuentas de capacidad, ubicación, reserva y lote se actualicen como una unidad coherente.

7.5 Reconsideración de órdenes

Cuando una recepción acepta al menos una unidad, `ReceivingService` invoca `WaitingOrderService.reconsider` El servicio vuelve a consultar disponibilidad para cada orden `WAITING_FOR_STOCK`; si todas sus líneas ahora tienen stock, la mueve a `QUEUED` y registra su nueva entrada a la cola. No se mueve una orden parcialmente abastecida si aún no puede cubrir todas las líneas requeridas.

También durante el reclamo el servicio vuelve a reservar las cantidades. Si otra orden consumió la disponibilidad entre la evaluación previa y la reclamación, la reserva devuelve nulo y la orden vuelve a esperar stock. Esto hace que la validación al ingresar no sea la única protección contra cambios concurrentes.

7.6 Cierre y resultados pendientes

Al detenerse el servicio, el consumidor es interrumpido y el método `stop` espera su terminación. Las recepciones ya aceptadas que no lleguen a almacenarse se registran como fallidas por cierre; la capacidad y la interrupción no convierten una orden en existencias. La espera del consumidor no tiene timeout, por lo que el método no retorna mientras el hilo siga vivo. Esto privilegia que el estado de recibos aceptados quede asentado, con el costo de que un cierre puede esperar si una dependencia interna no responde.

El sistema no guarda esos resultados en disco. Después de apagar la JVM, tanto la cola como el historial reciente desaparecen. El comportamiento de carga sostenida y cierre se describe a partir del código; requiere ejecución controlada para confirmar latencia y rendimiento en el entorno final.

8. Gestión de pedidos y políticas de planificación

8.1 Ingreso manual y cuentas

El formulario POST `/orders` recibe un nivel de servicio y una o más parejas de producto/cantidad. `OrderIntakeService` comprueba que existan listas, que sus tamaños coincidan, resuelve el producto en el catálogo y crea líneas con su tipo logístico. `OrderLine` rechaza cantidades menores que uno. El nivel se convierte al enum `ServiceLevel`; un valor ausente o inválido se comunica mediante mensaje de formulario.

El formulario deja seleccionar una de las cuentas conocidas. Para un alta nueva solicita nombre y correo. `CustomerRegistry` protege el registro con sincronización y deduplica por correo normalizado, por lo que volver a enviar un correo existente resuelve la cuenta ya registrada. El catálogo de clientes existe solo mientras la instancia está activa.

Al crear la orden, `WaitingOrderService` consulta todas las existencias necesarias. Si alcanza el stock, registra el estado `QUEUED` y `queueEnteredAt`; si no, registra `WAITING_FOR_STOCK` sin marcar la entrada a la cola procesable. La orden guarda el cliente, servicio, líneas, estado y marcas de tiempo.

8.2 Generador automático

`OrderGeneratorService.start` valida un intervalo de 1 a 3,600 segundos e inicia un hilo daemon, salvo que ya esté activo. El hilo duerme durante el intervalo, crea un pedido y repite. Escoge un nivel entre los cinco enums y selecciona cantidades pequeñas, de una a tres unidades si existe stock, o una unidad cuando no lo hay. Reutiliza clientes simulados por sus correos generados.

El generador intenta usar un producto encontrado en los lotes actuales; si no hay lotes elegibles inicia con `SKU-001`. Este comportamiento no garantiza una distribución uniforme de SKU ni de cantidades, y el generador puede producir pedidos que esperan inventario. Una excepción de ejecución se registra entre los errores recientes y termina el hilo; la interfaz permite volver a iniciarlo. No debe describirse como un generador ilimitado de carga con tasa fija garantizada: es una fuente didáctica con pausa configurable.

8.3 Cola elegible

`WaitingOrderService` mantiene el registro central y calcula el orden de `QUEUED` con un comparador de acuerdo con la política activa. Las órdenes sin stock se listan aparte. El despachador consulta esta cola y llama a `claimNext`, que intenta reservar stock antes de cambiar el estado a procesamiento. Así, una orden no consume un trabajador de procesamiento cuando ya se sabe que carece de existencias.

La cola no es una `BlockingQueue` separada: es una vista ordenada de órdenes almacenadas en un mapa sincronizado. El despachador revisa periódicamente si existe trabajo y espera brevemente si no hay orden disponible o si el pool alcanzó su límite. La distinción es importante para explicar la política real: se ordenan órdenes elegibles en el registro, y el hilo despachador las reclama.

El diagrama representa las rutas normales descritas en los servicios. La cola del pool también puede rechazar una tarea; en ese caso el despachador restaura la orden y vuelve a intentar luego. El estado `CANCELLED` existe en el enum, pero no se incluye como transición porque no se encontró un comando de cancelación expuesto.

8.4 Políticas disponibles

Política	Regla de orden	Desempate
Prioridad de servicio	Menor número de nivel primero: Exprés antes que Prioritario, Estándar, Programado y Económico.	Entrada a cola más antigua y luego ID.
Mayor tiempo de espera	Entrada a cola más antigua primero.	ID.
Menor carga de trabajo	Menor total de unidades primero.	Entrada a cola más antigua y luego ID.

“Mayor tiempo de espera” usa el momento de entrada a cola como FIFO, no una puntuación de envejecimiento combinada con prioridad. Las políticas no incluyen ponderaciones adicionales ni garantizan evitar inanición. La prioridad estricta puede demorar órdenes de menor nivel si llegan continuamente órdenes superiores.

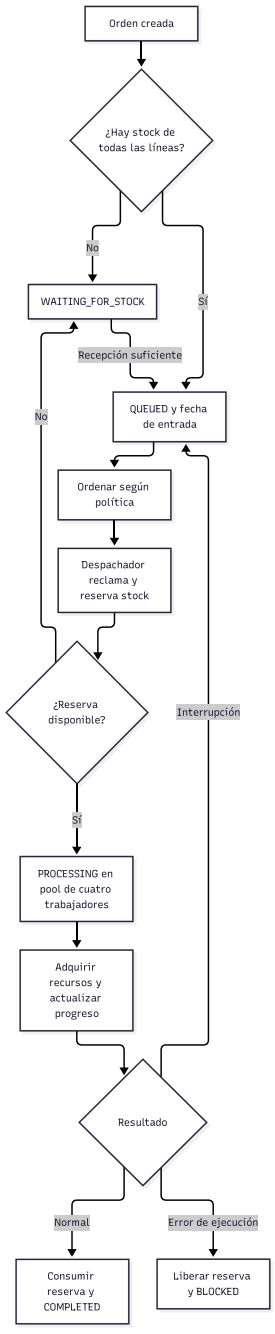


Figura 3: 8. Gestión de pedidos y políticas de planificación

8.5 Estados y recuperación

Una orden procesable puede pasar por `QUEUED`, `PROCESSING` y `COMPLETED`. Si no hay stock pasa por `WAITING_FOR_STOCK` y regresa a `QUEUED` tras una recepción suficiente. Una excepción de ejecución durante el procesamiento intenta cambiarla a `BLOCKED`, libera la reserva y registra un error. Una interrupción busca regresar la orden a cola y liberar la reserva en `finally`.

Si falla el envío de una tarea al pool, el despachador restaura la reclamación antes de reintentar. Las transiciones de estado se realizan dentro de métodos sincronizados de `WaitingOrderService`; la reserva y liberación del inventario también se coordinan con métodos sincronizados de `InventoryService`. La revisión se basa en la ruta normal del código, no en resultados de pruebas de estrés.

8.6 Métricas de espera

El tiempo de espera real se mide desde `queueEnteredAt` hasta el inicio del trabajador. El modelo almacena la duración y compara con el objetivo de servicio. La estadística de espera media incluye pedidos cuyo tiempo de espera ya se midió; los pedidos en espera no se incorporan como medición completada. El promedio de procesamiento se calcula con pedidos que tienen inicio y finalización.

Los objetivos son metas informativas, no temporizadores de expiración ni reglas de cancelación. El nivel Exprés tiene cero minutos; su medición puede figurar como incumplida si incluso el inicio ocurre después del instante de entrada. La interfaz indica pendiente mientras aún no existe una medición.

8.7 Volumen de trabajo

El costo simulado es `max(1, unidades) × 500 ms`, dividido en pasos de hasta 100 ms para actualizar el progreso. Por ello un pedido grande mantiene sus recursos más tiempo que uno pequeño. El tiempo restante se redondea a segundos hacia arriba. La simulación de trabajo no implica que cada unidad represente una operación física exacta; es una escala configurable en código para hacer observable la concurrencia en una demostración.

9. Concurrencia, recursos compartidos y prevención de deadlock

9.1 Procesamiento en paralelo

`OrderProcessingService` crea un `ThreadPoolExecutor` fijo de cuatro trabajadores, con una cola de tareas de capacidad 100 y rechazo explícito al llenarse. Un hilo despachador reclama órdenes y las envía al pool. El conjunto activo más cola se mantiene acotado: cuando alcanza el máximo, el despachador pausa brevemente en lugar de seguir reservando trabajo sin capacidad.

Cada reclamación ya lleva una `InventoryReservation`. El trabajador adquiere los recursos requeridos por las líneas, marca el inicio, simula el trabajo y actualiza porcentaje y segundos restantes. Al final completa la reserva y cambia estado. El diseño limita concurrencia por dos niveles distintos: máximo de cuatro trabajos activos en Java y capacidades propias para cada recurso operativo.

9.2 Semáforos de capacidad

`SharedResourceService` construye un semáforo justo por recurso, con tantos permisos como unidades configuradas. Fairness de `Semaphore` ordena solicitantes que esperan el mismo semáforo; no impone prioridad de nivel de servicio entre las órdenes globales. El servicio mantiene además un conjunto de titulares por código, usado para mostrar identificadores en el dashboard.

La adquisición devuelve un `Lease AutoCloseable`. El trabajador lo cierra en `finally`; el cierre es idempotente. Si una adquisición parcial se interrumpe o falla por código no configurado, el servicio libera los permisos que ya había conseguido antes de propagar la excepción. Los permisos se liberan en orden inverso de adquisición.

9.3 Orden total de adquisición

Antes de tomar permisos, `SharedResourceService` elimina códigos duplicados y ordena los códigos lexicográficamente. Toda orden usa el mismo orden global, aunque sus mercancías originales hayan enumerado los recursos en orden distinto. Si dos órdenes requieren recursos comunes, ninguna puede retener primero el segundo recurso mientras espera el primero cuando otra hace la operación inversa.

Esta estrategia elimina la condición de espera circular para adquisiciones productivas que pasan por este servicio, siempre que todos los participantes respeten la misma regla y que los permisos se liberen. No elimina el bloqueo indefinido causado por un hilo que no responde ni cubre recursos externos no modelados. La adquisición usa `Semaphore.acquire()` interrumpible, y las rutas de recuperación atienden interrupción.

9.4 Recursos que compiten en la operación

La combinación Voluminosa y Paletizada necesita simultáneamente montacargas y personal de bodega. En procesamiento normal las órdenes toman ambos recursos, con el orden común calculado por código, y retienen los permisos mientras trabajan. Esta contención es real para las órdenes productivas. La capacidad global permite más de una orden de cada tipo, hasta agotar las tres carretillas, diez encargados o el límite de cuatro trabajadores.

Electrónica y recepción coinciden en el escáner. La recepción adquiere `BARCODE_SCANNER` mediante `SharedResourceService`; el servicio de procesamiento lo incluye entre los códigos de Electrónica. Como ambos llaman al mismo semáforo de capacidad uno, no pueden ocupar simultáneamente el escáner. El lease del recibo se cierra al salir del bloque `try-with-resources`, incluidos caminos de excepción.

9.5 Monitores para estado compuesto

Los métodos sincronizados de `InventoryService` actualizan juntas cantidades por lote, disponibilidad, reservas, ocupación de ubicaciones y total almacenado. Los de `WaitingOrderService` protegen el mapa de órdenes y transiciones. La reclamación entra al servicio de inventario mientras mantiene el monitor de órdenes; el código revisado no adquiere en sentido inverso el monitor de órdenes desde un método del inventario, lo que evita ese ciclo de locks en estas rutas.

`CustomerRegistry` protege su mapa de clientes y correo. `ResourceConflictService` sincroniza sus cambios de estado explícito. Las listas de errores y resultados recientes usan `CopyOnWriteArrayList`, opción simple para snapshots pequeños. No se afirma que esta mezcla

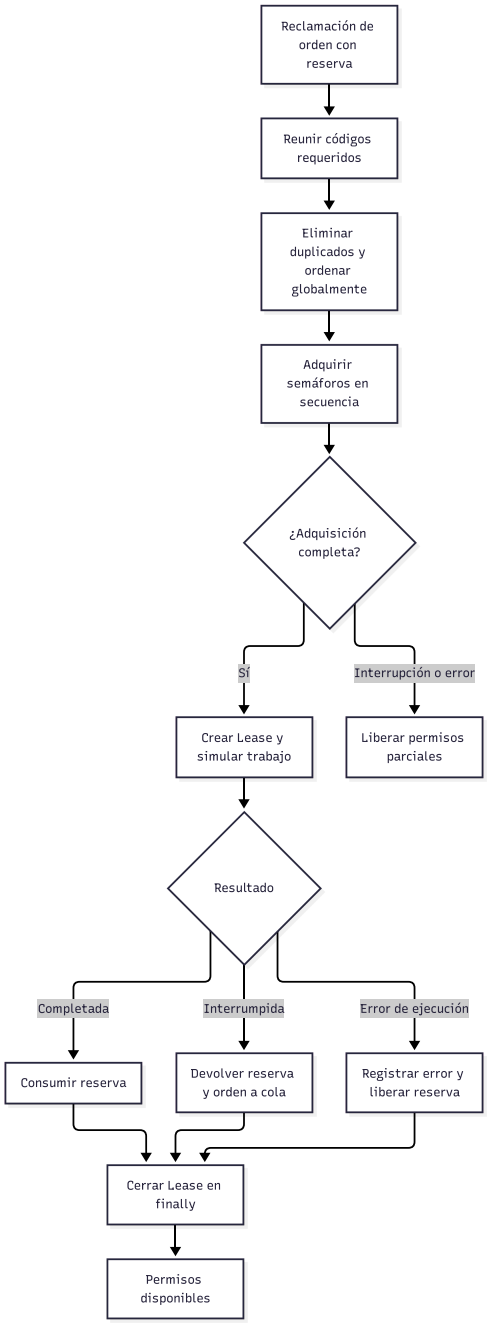


Figura 4: 9. Concurrencia, recursos compartidos y prevención de deadlock

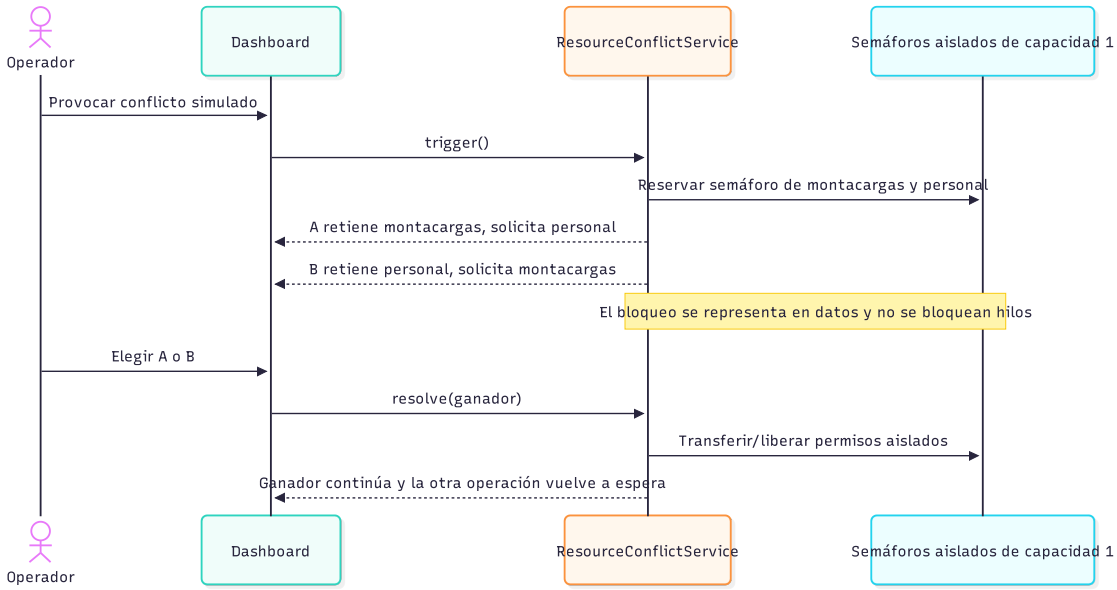


Figura 5: 10. Demostración de conflicto y manejo de errores

sea óptima para cargas grandes; responde al tamaño de una simulación local.

9.6 Límites de las garantías

La prevención por orden global aplica a `SharedResourceService`. El conflicto demostrativo usa sus propios semáforos y está fuera de dicho espacio. Si nuevas operaciones se agregan y toman recursos directamente o en otro orden, deben integrarse a la misma política para conservar la garantía. La prueba formal del orden de adquisición es una propiedad de diseño; el rendimiento real, equidad global y ausencia de defectos requieren pruebas de concurrencia que no se ejecutaron para elaborar este manual.

10. Demostración de conflicto y manejo de errores

10.1 Objetivo didáctico del conflicto

La asignación solicita que la interfaz pueda mostrar dos operaciones que retienen recursos en orden inverso y permita decidir cuál continúa. REDXela ofrece esa interacción mediante `ResourceConflictService`, pero la representa como estado explícito controlado, no mediante dos hilos que queden realmente bloqueados. Esta decisión evita congelar el servidor o agotar permisos productivos al tiempo que permite explicar la espera circular.

Las operaciones son “traslado voluminoso” y “preparación paletizada”. La primera aparece sosteniendo un montacargas y solicitando personal de bodega; la segunda, sosteniendo personal y solicitando montacargas. Es la misma pareja de recursos que comparten las categorías de mercancía Voluminosa y Paletizada en la operación normal.

10.2 Aislamiento y expiración

El servicio posee dos semáforos propios, separados de los mapas de permisos de `SharedResourceService`. En consecuencia, el escenario no reduce la disponibilidad real de órdenes ni figura como titular de recursos productivos. La palabra "bloqueada" es el estado mostrado de cada operación simulada; no existe un hilo detenido a la espera de un permiso.

El estado está protegido por métodos sincronizados. Si ya existe un conflicto activo, un nuevo clic no crea otro: devuelve el mismo estado con un mensaje que pide resolver o cancelar. El conflicto vence automáticamente después de 120 segundos mediante un `ScheduledExecutorService`; el usuario también lo puede cancelar antes. La expiración y el cierre liberan los semáforos aislados.

Al seleccionar A o B, el servicio hace una transferencia segura de los permisos representados, actualiza el estado del ganador y de la operación devuelta y marca el escenario resuelto. El "trabajo" del ganador se considera continuado y liberado dentro de la resolución: no se encola una operación de almacén real ni se añade una orden real a la cola. El texto de la UI y del manual debe conservar esta precisión.

10.3 Validación de formularios

`DashboardController` recibe números como texto para cantidades y convierte dentro de bloques que manejan `IllegalArgumentException`. Los parámetros de pedido y recepción pueden faltar; el controlador detecta campos vacíos, catálogo inválido, enum incorrecto o números mal formados y redirige al tablero con un mensaje flash. Los modelos de `OrderLine` y `ReceiptRequest` validan cantidades positivas. La recepción rechazada por cola llena o servicio detenido devuelve un estado y mensaje en vez de reportarse como encolada.

La validación del navegador (`required`, `min`, `type=email`) es una ayuda de interfaz; la validación del servidor protege el estado. El usuario puede manipular directamente las solicitudes, por lo que catálogos, identificadores y números se vuelven a resolver o validar en el backend. No se debe afirmar que exista protección CSRF personalizada, autenticación o autorización: no se encontraron esos componentes en el código inspeccionado.

10.4 Errores de procesamiento

`OrderProcessingService` mantiene hasta 20 errores recientes. Un error de ejecución en una orden se registra con fuente, ID de orden, mensaje y hora; intenta liberar la reserva y marcar la orden como `BLOCKED`. Si la recuperación falla, registra otro evento de fuente "Recuperación". Una interrupción se trata como señal de cierre o cancelación del trabajo: el `finally` libera el lease y devuelve la reserva/orden a cola, salvo que ya se haya asentado un resultado.

El despachador registra errores de envío al pool y restaura la reclamación. `OrderGeneratorService` también mantiene errores recientes; una excepción de ejecución detiene el generador y queda visible en la lista común del dashboard. Es una conducta deliberada para que una condición permanente no se reintente indefinidamente en un ciclo rápido.

Los resultados de recepción se exponen por separado y muestran aceptación, aceptación parcial, rechazo o fallo con detalle. La lista de resultados está limitada a 200 elementos. No todos los problemas de cada servicio se agregan a una única bitácora durable: errores de recepción se observan en sus resultados, mientras el panel de errores recientes une los registros del procesamiento y del generador.

10.5 Consideraciones de seguridad y operación

La interfaz está pensada para una demostración local. No se encontró inicio de sesión ni autorización por rol. Las formas aceptan comandos POST y la página carga Bootstrap desde un CDN. Antes de exponer la aplicación en una red pública se necesitaría un análisis de seguridad y controles que están fuera del alcance actual.

El mensaje visible es la forma primaria de retroalimentación; no sustituye logs persistentes ni alertas. Al reiniciar la JVM desaparecen órdenes, métricas, fallos recientes y estado del conflicto. Para una evaluación académica conviene observar los errores en el tablero y conservar la salida de ejecución cuando se necesite diagnóstico.

11. Dashboard, métricas y comparación de políticas

11.1 Vista principal

La página `dashboard.html` es una vista Thymeleaf en español. El controlador llena el modelo inicial con datos de catálogos y servicios. La página reúne tarjetas de estado, pedidos activos y completados, errores recientes, conflicto, formularios, comparación de políticas, cola de espera, stock por producto, lotes, resultados de recepción, recursos compartidos y requerimientos por mercancía.

La vista inicial sirve contenido legible incluso antes del primer refresco de JavaScript. La lógica dinámica consulta `/dashboard/state` cada 2.5 segundos, usando `fetch` y caché deshabilitada, y actualiza las áreas identificadas por sus IDs. No se reconstruye la página completa, lo que evita borrar normalmente la edición actual de formularios. Si la solicitud falla, el bloque captura la excepción y el siguiente intervalo vuelve a intentarlo sin cambiar los formularios.

La frecuencia de 2.5 segundos es una decisión de presentación; no convierte el sistema en tiempo real estricto. Entre dos sondeos puede existir un estado más nuevo en el servidor. Tampoco se agregan WebSockets o Server-Sent Events; el navegador consulta periódicamente.

11.2 Visibilidad disponible

El tablero permite consultar:

- cantidad total de órdenes, completadas, en cola, esperando stock y unidades registradas;
- promedio observado de espera hasta inicio y de procesamiento, más objetivos cumplidos/evaluados;
- órdenes activas, porcentaje de progreso, inicio, restante estimado y resultado de objetivo de espera;
- cola elegible en el orden de la política y lista separada de órdenes que esperan existencias;
- capacidad y uso de los seis recursos, con los IDs de titulares;
- existencias físicas por ubicación y por producto, unidades disponibles y reservadas, y lotes con asignación;
- recibos pendientes y resultados recientes;
- estado y explicación de la demostración simulada;
- comparación calculada y errores recientes del generador/procesamiento.

La estadística de órdenes completadas no significa ocupación porcentual histórica. El tablero enseña capacidad y unidades en uso por recurso, con lo que la fracción se puede derivar manualmente; no presenta una serie temporal. No se encontraron gráficas ni persistencia histórica.

Algunas zonas de control de generador se renderizan en la página inicial, pero el JSON vivo no incluye explícitamente estado e intervalo del generador; no se debe dar por hecho que todo ese control cambia sin recargar.

11.3 Datos y cálculo de las estadísticas

`WaitingOrderService.statistics()` cuenta órdenes y unidades del registro. La espera media usa órdenes con inicio medido; el promedio de procesamiento usa órdenes que tienen inicio y finalización. El contador SLA compara cada espera medida con los minutos del nivel. Por ello las métricas evolucionan cuando llegan órdenes a inicio o completan; una orden en cola todavía no aporta una medición real de espera.

El progreso de una orden está asociado con tiempo simulado. Cada 100 ms como máximo se actualiza el porcentaje y el restante. La duración total estimada depende de unidades, no de nivel de servicio. El tiempo restante es estimado y puede variar con la planificación y los recursos retenidos por otras órdenes.

Los recursos son snapshots de `SharedResourceService`: capacidad configurada, cantidad ocupada y titulares. El porcentaje de ocupación instantánea puede derivarse como `en uso / capacidad`; el backend no expone una estadística agregada de utilización durante un intervalo. La vista de inventario es más granular: separa disponible, reservado, almacenado y ubicaciones.

11.4 Comparación controlada

Al pulsar “Comparar políticas con la cola actual”, el controlador captura `findWaitingQueue()` una sola vez y entrega esa lista a `SchedulingComparisonService`. La proyección simula tres políticas sobre los mismos pedidos, cuatro trabajadores virtuales y el mismo costo de 0.5 segundos por unidad. Para cada política calcula promedio de espera total proyectada, procesamiento, finalización desde cola y objetivos cumplidos.

La cola capturada contiene pedidos elegibles al momento del comando, no pedidos `WAITING_FOR_STOCK`, activos o pedidos futuros. El cálculo incorpora el tiempo ya transcurrido desde `queueEnteredAt` como espera previa y asigna cada nuevo trabajo al trabajador virtual que queda disponible primero. Las comparaciones parten del mismo snapshot, por lo que las diferencias de orden no se confunden con listas de entrada distintas.

La proyección no ejecuta operaciones concurrentes reales ni reserva inventario, no simula los semáforos y no modela variación por mercancía. El promedio de procesamiento depende solo de las unidades y será igual entre políticas para una misma carga. Las diferencias principales surgen del orden y de la asignación a trabajadores virtuales. Debe interpretarse como experimento controlado sencillo y no como benchmark de producción.

11.5 Lectura de resultados

Antes de comparar, permita que se acumulen pedidos elegibles en la cola. La tabla informa el instante y tamaño de la captura para que el lector sepa qué carga se evaluó. Si la cola está vacía, no hay comparación útil. Una nueva acción reemplaza el snapshot anterior; los resultados no se guardan como historial de experimentos.

El porcentaje de objetivos cumplidos no está expresado como porcentaje en la vista, sino como cantidad cumplida sobre evaluada. Para valorar una política también conviene mirar

simultáneamente promedio de espera y finalización; mejorar uno puede perjudicar otro. Una política de menor carga puede favorecer pedidos chicos y retrasar grandes. Prioridad estricta puede relegar servicios inferiores. FIFO favorece antigüedad sin distinguir nivel.

11.6 Interpretación operacional

El tablero es un instrumento de observación de estado local. Un pedido no visible en la cola procesable puede estar esperando stock o en estado bloqueado; las listas no deben confundirse. Un recibo recién aceptado puede seguir en cola y todavía no cambiar el stock. Un conflicto activo no debe leerse como evidencia de que la producción real se congeló. Estas distinciones evitan conclusiones erróneas durante la exposición.

La actualización periódica consulta un objeto que se arma desde varios servicios en secuencia. Cada servicio mantiene su consistencia interna, pero no existe una transacción única que congele simultáneamente toda la JVM para responder el snapshot. Por eso una lectura puede combinar valores de instantes ligeramente distintos bajo actividad concurrente; para la escala de la simulación es una vista operacional y no un informe contable transaccional.

12. Ciclo de vida, límites y estado de verificación

12.1 Inicio de hilos y ejecutores

Spring crea los beans de servicio. En `@PostConstruct`, `ReceivingService` pone en ejecución su consumidor daemon y `OrderProcessingService` inicia el despachador daemon. El pool fijo de cuatro trabajadores se crea como parte del servicio de procesamiento y crea sus hilos cuando recibe tareas. `OrderGeneratorService` crea su hilo al recibir el comando de inicio. `ResourceConflictService` mantiene un planificador daemon dedicado a expirar una simulación activa.

El orden exacto de invocación de métodos `@PreDestroy` depende del ciclo de vida del contexto Spring y de sus dependencias. Cada servicio contiene su propio cierre; no hay un coordinador de shutdown adicional que persista el estado o drene datos hacia almacenamiento durable.

12.2 Cierre del generador

`OrderGeneratorService.stop()` interrumpe el hilo y espera a que termine antes de limpiar su referencia. Si quien invoca el cierre recibe interrupción, conserva ese estado, reinterrumpe el generador y continúa esperando. El bucle termina al salir del `sleep` o cuando la generación retorna y el hilo observa la interrupción.

La espera del generador no tiene timeout. Como consecuencia, `stop()` puede esperar indefinidamente si una operación síncrona usada por `generateOne()` se bloquea y no responde a interrupciones. En el flujo actual la generación ejecuta consultas en memoria y creación de pedido; aun así, el manual no afirma una cota temporal formal.

12.3 Cierre del consumidor de recepción

`ReceivingService.stop()` marca el servicio detenido, interrumpe al consumidor y espera mientras permanezca vivo. La cola de elementos aceptados se sigue recorriendo; al detectar que el servicio se detuvo, las solicitudes no procesadas se convierten en resultados fallidos de cierre, en

vez de dejarse como aceptadas sin explicación. El consumidor puede terminar cuando ya no queda trabajo. La espera también carece de timeout.

La interrupción mientras se espera por el escáner se captura y produce un fallo visible para el recibo. El `try-with-resources` cierra el lease si ya se adquirió. Estas propiedades se desprenden de las rutas de código inspeccionadas; no se sometieron a una prueba concurrente durante la preparación de este texto.

12.4 Cierre de despacho y trabajadores

`OrderProcessingService.shutdown()` cambia la bandera de ejecución, interrumpe el despachador y espera hasta dos segundos para que termine. Si sigue vivo, registra una condición de fallo. Luego realiza cierre ordenado del pool y espera cinco segundos. Si no finaliza, invoca `shutdownNow`, restaura en cola las tareas aún no iniciadas y espera un periodo forzado adicional de cinco segundos. Si los trabajadores continúan, lanza `IllegalStateException` en vez de informar silenciosamente éxito.

Los trabajadores activos reciben interrupción. Su ruta de `finally` cierra permisos y devuelve a cola la reserva si no se había asentado. La restauración de tareas pendientes es idempotente por `OrderTask.restore()`. Si el hilo que cierra es interrumpido, el servicio conserva el estado de interrupción después del manejo de tareas.

Los límites de tiempo son finitos para el pool; la parada del hilo generador y el consumidor esperan sin plazo. La terminación real todavía depende de que el código activo responda a interrupción o complete su operación. Un informe de fallo no mata a la fuerza un hilo que ignore la interrupción.

12.5 Persistencia y límites de operación

Todo el estado de negocio es volátil: órdenes, cuentas nuevas, historial de recibos, lotes, reservas, estadísticas y errores recientes desaparecen con el proceso. No se ofrece restauración tras fallo, transacción distribuida ni coordinación entre instancias. Ejecutar dos procesos crea dos simulaciones independientes.

Las restricciones de recursos se respetan durante el procesamiento normal mediante semáforos. El conflicto de demostración utiliza semáforos aislados y estado explícito, sin hilos realmente bloqueados. La comparación de políticas es una proyección de la misma carga capturada, no un benchmark con recursos reales. El uso de Bootstrap requiere acceso al CDN. El tablero no incluye autenticación ni control de roles.

La aplicación informa ocupación instantánea, pero no almacena series temporales. La lista de errores es reciente y acotada: hasta 20 errores visibles de generador y procesamiento, además de hasta 200 resultados de recepción. No es un sistema de observabilidad durable.

12.6 Estado de verificación

La documentación se redacta mediante inspección de fuentes y configuración. Se consideran confirmados por código: las capacidades declaradas, los catálogos, los métodos de sincronización, las rutas de formulario, las propiedades de cola, el costo de simulación y el manejo de cierre que aparece en las clases citadas. No se afirma que se ejecutaron pruebas automatizadas, pruebas de carga, ensayos de deadlock o verificación visual del PDF.

Antes de entregar, conviene completar una verificación reproducible en el entorno objetivo: compilar con JDK 25; iniciar la aplicación; probar formularios válidos e inválidos; observar una recepción parcial y reconsideración de órdenes; confirmar contención de escáner y recursos; ejecutar y resolver/cancelar conflicto; detener el sistema con actividad; generar el PDF y revisar paginación, enlaces y diagramas. Esta lista es recomendación de verificación pendiente, no un resultado ya aprobado.

12.7 Limitaciones que deben exponerse

Las simplificaciones deben presentarse con claridad durante la defensa:

1. El procesamiento usa una duración fija por unidad, no tiempos medidos de operación.
2. La comparación omite contención real y solo representa cuatro trabajadores virtuales.
3. La demostración de espera circular es visual y aislada; no detiene la cola real.
4. No hay almacenamiento persistente ni recuperación tras reinicio.
5. La tasa del generador es configurable, pero la variedad de productos y cantidades depende del stock observado y de reglas simples.
6. La vista periódica puede tener hasta un intervalo de retraso y no es una captura transaccional global.

Con esos límites explícitos, el sistema sirve como artefacto didáctico para explicar coordinación, exclusión mutua, colas, prioridades y ciclo de vida de recursos sin presentar una simulación pequeña como sistema industrial terminado.